

Practical extraction and reporting language (Perl)

Nathan Warner



Northern Illinois
University

Computer Science
Northern Illinois University
United States

Contents

1	Perl	2
2	Comments	4
3	Command line options	5
4	Variables	6
5	Arrays	8
6	Hashes	11
7	Special variables	13
8	More on strings	16
9	Operators and functions	17
9.1	Quote operators and siblings	19
9.2	Important builtin functions	20
10	Constructs	22
11	Input / output	26
12	Subroutines	28
13	Regular expressions	30
14	Idioms	33
15	Perl	34

Perl

- **Practical extraction and reporting language (perl):** Perl is a high-level, general-purpose, interpreted, dynamic programming language. Though Perl is not officially an acronym, **Practical Extraction and Reporting Language** is commonly used as a **backronym**.
- **use version pragma:** The use VERSION pragma in Perl enforces a minimum version required to run a script and automatically enables modern language features

You should place this declaration at the very top of your file. Modern Perl code uses the vX.Y v-string format rather than old-style decimals:

```
0 use v5.36
```

Versions v5.10 and higher automatically activate the feature pragma for that release. For example, use v5.36; is equivalent to loading use feature ':5.36';. This gives you immediate access to built-in keywords like say, state, and updated signatures without typing extra code

Declaring use v5.11 or higher automatically turns on the strict pragma (use strict;) lexically across your file.

Declaring use v5.35 or higher automatically turns on the warnings pragma (use warnings;

- **use feature:** In Perl, the use feature pragma allows you to enable newer syntactic features and capabilities without breaking backward compatibility for older programs. Because it is a lexical pragma, any feature you enable is only active from the line it is declared until the end of its enclosing code block.
 - **Individual Features:** Pass a list of feature names to the pragma.

```
0 use feature 'say';
1 say "Hello, world!"; # Appends a newline
```

- **Feature bundles:** Load all features made available in a specific Perl release by prefixing the version with a colon

```
0 use feature ':5.36'
```

- **Implicit loading:** Requiring a minimum Perl version automatically enables the feature bundle for that version
- **Command Line Execution:** Use the -E flag instead of -e to quickly test code snippets with all features enabled for your current Perl version.

```
0 perl -E 'say "Fast one-liner testing!'"
```

- **Using the diagnostics pragma:** Another pragma that you may find useful is the diagnostics pragma. This translates the normally terse diagnostics emitted from the perl compiler and the perl interpreter into much more useful ones

```
0 use diagnostics;
```

The diagnostics pragma makes your warnings much more verbose, and it slows the start-up time of your script considerably. You should remove it before putting your code into production.

- **Memory management:** Perl 5 memory management is primarily based on reference counting, with a runtime that manages Perl values such as scalars, arrays, hashes, and objects for you.

Comments

- **Standard comments:** Uses a hashtag

```
0  # Comment
```

- **Block comments:** We can also use Plain Old Documentation (POD)

```
0  =begin comment
1  ...
2  ...
3  =end comment
4
5  =cut
```

=cut resumes Perl code execution. "Stop ignoring this text; switch back to executable Perl code."

=end comment closes a =begin comment region. "This specific comment block is done, but continue treating subsequent text as documentation."

- **__END__**: You can signal to Perl the end of your program by using the special **__END__** tag. Anything below **__END__** will be ignored by Perl.

```
0  #!/usr/bin/perl
1
2  print "Hello world";
3
4  __END__
```

Command line options

- **Warnings:** Turn on warnings with -w

```
o  #!/usr/bin/perl -w
```

Variables

- **Pragmatic information (pragma):** A code pragma (short for "pragmatic information") is a special instruction or command embedded in source code. It tells a compiler, interpreter, or assembler how to process the program, but it does not change the actual logic or output of the software itself.
- **Note about variables:** In Perl, it is not necessary to declare your variables before you begin. You can summon a variable into existence simply by using it, and it will be globally available to any routine in your program. If you're used to programming in C or any of a number of other languages, this may seem odd and even dangerous to you. This is indeed the case. That's why you want to use the strict pragma.
- **Variable sigils (prefixes):** In Perl, variable prefixes are called sigils, and they immediately identify the data type of the variable.
 - \$ used for scalars
 - @ used for arrays
 - % used for hashes. A hash is an unordered collection of key-value pairs (associative arrays).
- **my, our, and state:** In Perl, my declares a truly private lexical variable, state creates a persistent static variable that initializes only once, and our creates a temporary shortcut to a global package variable

A lexical variable is a variable that is available only within a specific, textually defined part of a program, such as a function, a block, or a file.

- **The strict pragma:** Once we use the strict pragma, we have to explicitly declare new variables with scope keywords my, our, or state.

```
0  #!/usr/bin/perl -w
1  use strict;
2
3  my $x = 50; # Okay
4  $y = 100; # Bad
```

- **Variables with name a or b:** \$a and \$b are special variables in Perl. they are implicitly used by Perl's built-in sort function. Inside the sort block, Perl automatically places the two values being compared into \$a and \$b. Consequently, strict 'vars' specifically permits \$a and \$b without requiring my
- **Scalars:** A scalar is a single item of data

```
0  my $name = "Arthur";
1  my $whoami = 'Just Another Perl Hacker';
2  my $meaning_of_life = 42;
3  my $number_less_than_1 = 0.000001;
4  my $very_large_number = 3.27e17; # 3.27 by 10 to the power of
   ↪ 17
```

A scalar can be text of any length, and numbers of any precision (machine dependent, of course). Perl doesn't need us to tell it what type of data we're going to put into the scalar. In fact, Perl doesn't care if the type of data in the scalar changes throughout your program. Perl magically converts between them when it needs to

- **Single vs double quotes:** In Perl, quotes are required to distinguish strings from the language's reserved words or other expressions. Either type of quote can be used, but there is one important difference: double quotes can include other variable names inside them, and those variables will then be interpolated

```
0 my $x = 50;  
1 print "$x";
```

- **Special characters:** Special characters, such as the `\n` newline character, are only available within double quotes. Single quotes will fail to expand these special characters just as they fail to expand variable names
- **String concatenation:** Uses the dot notation

```
0 print "Hello, " . "World!";
```

- **Bash style interpolation:**

```
0 my $x = "Hello";  
1 print "${x}, World";
```

- **References / dereferencing:**

```
0 my $x = 10;  
1 my $y = \ $x; # Creates a reference  
2  
3 $$y = 25; #
```

Dereferences

- `\$scalar`
- `\@array`
- `\%hash`
- `\&function`

Arrays

- **Creating arrays:** Arrays are initialised by creating a comma separated list of values:

```
0 my @arr = (1,2,3);
```

Perl's arrays, like those in many other languages, are indexed from zero

- **Accessing:** As you would expect

```
0 print $fruits[0];
```

We use the dollar sign here because the indexing produces a scalar.

- **Slicing:** If we wanted to only deal with a portion of the array, we use what we call an array slice. These are written as follows:

```
0 @arr # entire array
1 @arr[1,3]
2 @arr[0..2]
```

- **Negative indexing:**

```
0 print $arr[-1] # 3
```

- **Array size:**

```
0 print $#arr + 1;
```

Note that `$#arr` gets the index of the last element. We can also use

```
0 my $size = @arr;
1 print my $size = @arr;
```

- **qw// (quote words):** It takes whitespace separated words and turns them into a list

```
0 my @arr = qw/hello , world/
```

Your list can stretch over multiple lines, and your delimiter doesn't need to be a slash. Whatever punctuation character that you place after the `qw` becomes the delimiter. So if you prefer parentheses over slashes, that's no problem at all

- **Printing out arrays**

```

0 my @arr = (1,2,3);
1 print @arr;
2 print join(", ", @arr);
3 print "@arr"

```

The first method takes advantage of the fact that `print` takes a list of arguments to print and prints them out sequentially. The second uses `join()` which joins an array or list together by separating each element with the first argument. The third option uses double quote interpolation and a little bit of Perl magic to pick which character(s) to separate the words with.

- **Context:** There's a term you've heard used just recently but which hasn't been explained: context. Context refers to how an expression or variable is evaluated in Perl. The two main contexts are:
 - Scalar context
 - List context

Scalar variables are always evaluated in scalar context, however arrays and hashes can be evaluated in both scalar contexts (when we treat them as scalars) and list contexts (when we treat them as arrays and hashes). Scalar variables are always evaluated in scalar context, however arrays and hashes can be evaluated in both scalar contexts (when we treat them as scalars) and list contexts (when we treat them as arrays and hashes).

Many things in Perl are very specific about which context they require and will force lists into scalar context when required. For example the `+` (plus or addition) operator expects that its two arguments will be scalars. Hence:

```

0 my @a = (1,2,3);
1 my @b = (4,5,6,7);
2 print @a + @b;

```

will print 7. There's also a third type of context, the null context, where the result of an operation is just thrown away. This usually isn't discussed, because by its very definition we don't care about what result is returned

- **Lists:** A list is an unnamed array.

```

0 print ("Hello", ",", " ", "World", "\n");

```

If you come across something that wants a LIST, you can either give it the elements of list as in the first example above, or you can pass it an array by name. If you come across something that wants an ARRAY, you have to actually give it the name of an array. Examples of functions which insist on wanting an ARRAY are `push()` and `pop()`, which can be used for adding and removing elements from the end of an array

- **Anonymous array reference:**

```
0 my $ref = [1,2,3];
```

The `[ ... ]` syntax is used to create and assign an anonymous array reference. Square brackets `[ ]` create a memory reference to an array that has no name. This reference is a single scalar value, meaning it is stored in a standard scalar variable (`$`).

- **Multiple dimensional arrays:**

```
0 my $matrix = [  
1     [1,2,3],  
2     [4,5,6],  
3     [7,8,9]  
4 ];
```

Passing a reference to a subroutine avoids flattening your data structure and is much faster for large lists because Perl only copies the single memory address rather than the entire list.

- **Dereferencing:**

```
0 print $array_ref->[0]; # Outputs: apple  
1  
2 # Loop through all elements  
3 for my $item (@$array_ref) {  
4     print "$item\n";  
5 }
```

The syntax

```
0 @{$array_ref}
```

is used to dereference an array reference.

Hashes

- **Hashes:** A hash is a two-dimensional array which contains keys and values, they're sometimes called "associative arrays", or "lookup tables". Instead of looking up items in a hash by an array index, you can look up values by their keys.

Hashes are initialised in exactly the same way as arrays, with a comma separated list of values:

```
0 my %monthdays = ("January", 31, "February", 28, "March", 31,  
  ↪ ...);
```

Or,

```
0 my %monthdays = (  
1   "January" => 31,  
2   "February" => 28,  
3   "March" => 31,  
4   ...  
5 );
```

The => operator is syntactically the same as the comma, but is used to distinguish hashes more easily from normal arrays. It's pronounced "fat comma", or less often "fat arrow". It does have one difference, you don't need to put quotes around a bare word immediately before the => operator as these are always treated as strings:

- **Reading hash values:**

```
0 print $monthdays{"January"}; # prints 31
```

Bare words inside the braces of the hash-lookup will be interpreted in Perl as strings, so usually you can drop the quotes:

- **Creating hash elements:** You can also create elements in a hash on the fly:

```
0 my %monthdays = ();  
1 $monthdays{"January"} = 31;  
2 $monthdays{February} = 28;  
3 ...
```

- **Changing hash values:** To change a value in a hash, just assign the new value to your key:

```
0 $pizza_prices{small} = '$6.00'; # Small pizza prices have  
  ↪ gone up
```

- **Deleting hash values:** Use the delete function

```
0 delete($pizza_prices{medium}); # No medium pizzas anymore.
```

- **Finding out the size of a hash:** Strictly speaking there is no equivalent to using an array in a scalar context to get the size of a hash. If you take a hash in a scalar context you get back the number of buckets used in the hash, or zero if it is empty. This is only really useful to determine whether or not there are any items in the hash, not how many.

If you want to know the size of a hash, the number of key-value pairs, you can use the `keys` function in a scalar context. The `keys` function returns a list of all the keys in the hash.

```
0 my $size = keys %monthdays;  
1 print $size;
```

Special variables

- **Intro:** Perl has many special variables. These are used to set or retrieve certain values which affect the way your program runs. For instance, you can set a special variable to turn interpreter warnings on and off (`$^W`), or read a special variable to find out the command line arguments passed to your script (`@ARGV`).
- `$_`: Represents the current thing that your Perl script's working with — often a line of text or an element of a list or hash. It can be set explicitly, or it can be set implicitly by certain looping constructs (which we'll look at later).

The special variable `$_` is often the default argument for functions in Perl. For instance, the `print()` function defaults to printing `$_`.

```
0  $_ = "Hello, world!\n";
1  print;
```

- **@ARGV:** Command line args end up in this special array.

```
0  # ./main.pl arg1 arg2
1  print "@{ARGV}" arg1 arg2
2  print my $argc = @ARGV; 2
```

- **%ENV:** This hash contains the names and values of environment variables.
- **!:** `!` contains the operating-system error corresponding roughly to `errno` in C.

```
0  open(my $fh, "<", "foo.txt")
1      or die "open failed: !";
```

- **Input record separator `$/`:** Normally,

```
0  $/ = "\n";
```

- **Proc ID `$$`:** When used by itself without a variable name, `$$` is a predefined global variable that returns the Process ID (PID) of the currently running Perl script.
- **List of general special variables:**
 - `$_` Default input / working variable
 - `@_` Arguments passed to the current subroutine
 - `$"` Separator used when interpolating arrays into strings
 - `$$` Current process ID
 - `$0` Name of the running Perl program
 - `$(` Real group ID
 - `$)` Effective group ID
 - `$<` Real user ID

- \$> Effective user ID
- \$; Multidimensional array subscript separator; largely obsolete
- \$a, \$b Special comparison variables used by sort
- %ENV Process environment variables
- \$] Perl version as a numeric value
- \$^V Perl version as a version object
- \$^O Operating system Perl was built for
- \$^X Perl executable used to start the program
- \$^T Program start time
- \$^F Maximum system file descriptor
- \$^I In-place editing extension (-i)
- \$^M Emergency memory pool
- @F Autosplit fields when using -a
- @INC Module/library search paths
- %INC Files/modules already loaded
- \$INC Current @INC hook index in newer Perl versions
- @ISA Parent classes of a package
- %SIG Signal handlers
- %{\^HOOK} Hooks for certain Perl operations
- \${^MAX_NESTED_EVAL_BEGIN_BLOCKS} Limit for nested eval/BEGIN processing

- **Regex variables**

- \$1, \$2, \$3, ... Captured groups
- @{\^CAPTURE} All capture-buffer contents
- \$& Entire matched string
- \$' Text before the match
- \$' Text after the match
- \$+ Highest-numbered capture that matched
- \$^N Most recently closed capture
- @- Starting offsets of match/captures
- @+ Ending offsets of match/captures
- %- Named captures, including multiple captures with the same name
- %+ Named captures
- \${\^MATCH} Entire /p match
- \${\^PREMATCH} Text before a /p match
- \${\^POSTMATCH} Text after a /p match
- \${\^LAST_SUCCESSFUL_PATTERN} Last successful regex pattern
- \$^R Result of regex (?{ ... }) evaluation
- \${\^RE_COMPILE_RECURSION_LIMIT} Regex compile nesting limit
- \${\^RE_DEBUG_FLAGS} Regex debugging flags
- \${\^RE_TRIE_MAXBUF} Regex trie optimization memory control

- **Command-line and filehandle variables:**
 - @ARGV Command-line arguments
 - \$ARGV Current filename being processed by <>
 - ARGV Special filehandle used by <>
 - ARGVOUT Output filehandle used by in-place editing
 - \$. Current input line number
 - \$/ Input record separator
 - \$ Output record separator
 - \$, Output field separator
 - \${^LAST_FH} Last filehandle read
- **Error and process-result variables:**
 - \$! OS/C-library error (errno)
 - %! Symbolic access to errno values
 - \$@ Exception from the most recent eval
 - \$? Exit/wait status of the last external process
 - \$^E Extended OS-specific error information
 - \${^CHILD_ERROR_NATIVE} Native child-process status
 - \$^S Interpreter/evaluation state
 - \$^W Global warning flag
 - \${^WARNING_BITS} Lexically enabled warning categories

More on strings

- **Foldcase:** case folding is performed using the built-in `fc` function (introduced in Perl 5.16). Case folding converts strings into a canonical format meant for case-insensitive comparisons, handling complex Unicode character mappings.

```
0 use feature 'fc'; # or use v5.16;
```

```
0 $a = 'H';  
1 $b = "h";  
2  
3 print fc($a) cmp fc($b); # 0
```

- **String escape sequences:**
 - `\u` (Uppercase next): Capitalizes only the very next character.
 - `\l` (Lowercase next): Lowers only the very next character.
 - `\U` (Uppercase all): Capitalizes all characters that follow it.
 - `\L` (Lowercase all): Lowers all characters that follow it.
 - `\F` (Foldcase all): Case-folds all characters that follow it (ideal for Unicode case-insensitive matching).
 - `\E` (End modification): Stops the effect of `\U`, `\L`, or `\F`.

Operators and functions

- **Operators**

- **Arithmetic:** +, -, *, /, %, ** (exp), +=, -=, *=, /=, %, **=

Note: In fact, the shortcut operators += and its siblings can be used for just about any operator.

We also have post/pre increment/decrement.

- **Logical:** &&, and, ||, or, !, not, xor, //

Note: The versions and, or, not, xor have extremely low precedence compared to &&, ||, and !, lower even than the assignment operator =.

The // operator is defined-or, which returns the left side if it is defined, even if it is false (like 0 or "").

- **Numeric conditional:** ==, !=, <, >, <=, >=, <=>
- **String conditional:** eq, ne, lt, gt, le, ge, cmp

Note: <=> and cmp perform three-way sorting and evaluation. They return -1, 0, or 1 for less, equal, or greater.

- **String:** . (concat), x (repeat)

```
0  "hello" . "world";
1  "hello" x 3;
```

- **Ternary:** Uses C syntax

```
0  condition ? if_true : if_false
```

- **Function calling:** Perl does not insist that functions enclose their arguments within parentheses. Both print "Hello"; and print("Hello"); are correct.

- **Argument types:**

- **SCALAR** – Any scalar variable, for example: 42, "foo", or \$a.
- **EXPR** – An expression (possibly built out of terms and operators) which evaluates to a scalar.
- **LIST** – Any named or unnamed list (remember that a named list is an array).
- **ARRAY** – A named list; usually results in the array being modified.
- **HASH** – Any named or unnamed hash.
- **PATTERN** – A pattern to match on — we'll talk more about these later on, in Regular Expressions.
- **FILEHANDLE** – A filehandle indicating a file that you've opened or one of the pseudo-files that is automatically opened, such as STDIN, STDOUT, and STDERR.

- **Data conversion functions:**

- **hex:** Interprets a string as a hexadecimal number and converts it to decimal.
- **oct:** Interprets a string as an octal number and converts it to decimal. If the string starts with 0x, it automatically parses it as hex, or with 0b as binary.
- **int:** Strips the decimal portion of a floating-point number, truncating it to an integer.
- **chr:** Converts an ASCII/Unicode numeric value into its corresponding character.
- **ord:** The inverse of chr. Converts the first character of a string into its numeric ASCII/Unicode value.
- **join:** Flattens a list or array into a single string, using a specified separator string.

```
o my $string = join("-", ("a", "b", "c")); # Returns
  ↪ "a-b-c"
```

- **split:** The inverse of join. Breaks a string into a list of substrings using a regular expression delimiter.

```
o my @array = split(/-/ , "a-b-c"); # Returns ('a', 'b',
  ↪ 'c')
```

9.1 Quote operators and siblings

- **q// (Single Quote):** Behaves exactly like standard single quotes ('...'). It does not interpolate variables or escape sequences.
- **qq// (Double Quote):** Behaves exactly like standard double quotes ("..."). It interpolates variables, math, and escape sequences (like \n or \U).
- **qw// (Quote Words):** Splits a string on whitespace and returns a list of literal words. No variable interpolation occurs

9.2 Important builtin functions

- **Numeric:**

- `abs($x)` Absolute value
- `sqrt($x)` Square root
- `exp($x)` $e^{**} \$x$
- `log($x)` Natural logarithm
- `sin($x)` Sine
- `cos($x)` Cosine
- `atan2($y, $x)` Arctangent
- `rand($n)` Random number from 0 to $< \$n$
- `srand()` Seed random number generator

- **String:**

- `length($str)` String length
- `substr($str, ...)` Extract/replace substring
- `index($str, $sub)` Find substring
- `rindex($str, $sub)` Find substring from right
- `lc($str)` Lowercase entire string
- `uc($str)` Uppercase entire string
- `fc($str)` foldcase entire string
- `lcfirst($str)` Lowercase first character
- `ucfirst($str)` Uppercase first character
- `chomp($str)` Remove input record separator, usually newline
- `chop($str)` Remove final character
- `sprintf(...)` Produce formatted string

```
0  sprintf(FORMAT, LIST);  
1  
2  my $price = 19.9962;  
3  my $money = sprintf("%.2f", $price);
```

- `split(...)` Split string into list
- `join(...)` Combine list into string

- **Arrays:**

- `push(@a, $x)` Add to end
- `pop(@a)` Remove from end
- `unshift(@a, $x)` Add to beginning
- `shift(@a)` Remove from beginning
- `splice(@a, ...)` Insert/remove array elements
- `reverse(@a)` Reverse elements
- `sort(@a)` Sort elements
- `grep ... @a` Filter elements
- `map ... @a` Transform elements
- `join($sep, @a)` Convert array to string
- `scalar(@a)` Number of elements
- `each(@a)` Iterate index/value pairs on modern Perl

```

0  my @nums = (3, 1, 2);
1
2  push @nums, 4;
3
4  my $last = pop @nums;
5
6  my @sorted = sort { $a <=> $b } @nums;
7
8  my @even = grep { $_ % 2 == 0 } @nums;
9
10 my @doubled = map { $_ * 2 } @nums;

```

- **Hashes:**

- keys(%h) Get keys
- values(%h) Get values
- each(%h) Get key/value pairs while iterating
- exists(\$h{\$key}) Check whether key exists
- delete(\$h{\$key}) Remove key
- scalar(keys %h) Number of keys

Constructs

- If

```
0  if (cond) {
1      ...
2  }
3
4  elsif (cond) {
5      ...
6  }
7
8  else {
9      ...
10 }
```

- Unless: inverse of an if

```
0  unless (cond) {
1      ...
2  }
```

Note that the brackets are required, and there is no `elsunless`.

- Shorthand: If if or unless require a single statement, we can put the statement before the check

```
0  print "Hello" if 1;
```

- Existence and definitiveness: To find out if something is defined, use Perl's `defined` function. The `defined` function is necessary to distinguish between a value that is false because it is undefined and a value that is false but defined, such as 0 (zero) or "" (the empty string).

```
0  my $skippy; # skippy is undefined and false
1  $skippy = "bush kangaroo"; # true and defined
2  print "true" if $skippy; # prints true
3  print "defined" if defined($skippy); # prints defined
4
5  my $undef = undef;
6  print "defined" if defined($undef); # does not print
7  print "undefined" if not defined($undef); # prints undefined
```

- `undef`: When you pass a variable or expression to it, `undef` behaves as a function that un-defines the variable and frees its memory. The expression must be an lvalue

```
0  undef $scalar;          # Resets a scalar to the undefined state
```

When used on its own without any arguments, it simply returns the undefined value.

```
0 my $name = undef;    # Explicitly assigning the undefined
  ↪ value
```

- **While:**

```
0 while (cond) {
1     ...
2 }
```

- **Until:** The opposite of a while

```
0 until (cond) {
1     ...
2 }
```

- **Shorthand while / unless:** Similar to if and unless,

```
0 print "Hello" while 1;
1 print "Hello" until 0;
```

- **For:** Identical to C

```
0 for (my $i = 0; $i < $n; ++$i)
```

- **Foreach:**

```
0 foreach(@arr) {
1     print "$_\n";
2 }
3
4 print "${_}\n" foreach(@arr);
```

Note: When in a foreach loop, the variable representing the current element is the current element, not a copy of it.

- **Foreach range syntax:**

```
0 print "${_}" foreach (1..10)
```

- **For / foreach and variable creation**


```

0  for my $i (0..10) { ... }
1  foreach my $i ((1,2,3)) {...}

```

- **= do ...:**

```

0  my $variable = do { ... };

```

executes a block of code and returns the value of the last statement evaluated inside that block

- **eval BLOCK:**

```

0  eval {...}

```

acts like a try-catch block.

```

0  # Example of trapping a division by zero error
1  eval {
2      my $result = 10 / 0;
3  };
4
5  if ($?) {
6      print "Error caught: $@";
7  }

```

- **eval EXPR:** This form compiles code string text dynamically

```

0  # Example of dynamically executing a string
1  my $code = '$x = 5 + 5;';
2  eval $code;
3  print $x; # Outputs 10

```

- **Labeled blocks:** You can assign a name (called a label) to any code block by placing the name followed by a colon (:) right before the opening brace. This allows you to jump out of or control specific nested blocks using loop control statements like last, next, or redo.

Always write labels in ALL CAPS to ensure they stand out and never conflict with reserved Perl keywords.

```

0  OUTER_LOOP: while ($x < 10) {
1      INNER_LOOP: while ($y < 10) {
2          if ($condition) {
3              # Exits BOTH loops completely, not just the inner
3              ↪ one
4              last OUTER_LOOP;
5          }
6          $y++;
7      }
8      $x++;
9  }

```

- redo restarts the current iteration immediately.
- next skips to the next iteration
- last exits the loop completely (like break in C/Java).

Input / output

- **Predefined file handles:**

```
0  STDIN    # standard input
1  STDOUT   # standard output
2  STDERR   # standard error
```

- **<...> syntax:** Primarily a read operation on a filehandle

```
0  my $line = <STDIN>;
```

Read the next line from standard input.

- **Writing to file handle:**

```
0  print STDOUT "Hello\n";
```

Although STDOUT is the default.

- **readline:** Reads a line from a file handle

```
0  my $line = readline(STDIN);
```

Reads a line from

- **say and printf:** Say is essentially C puts; takes a single string argument and appends a newline before printing it.

```
0  use feature "say";
1  say "Hello, world";
2  printf "%s", "Hello, world";
```

- **Opening a file:**

```
0  open(HANDLE, MODE, PATH);
```

- < read
- > write, truncate existing file
- » append
- +< read and write (starts at beginning)
- +> read and write, truncate
- +» read and append (starts at end)

- **Closing a file:**

```
0  close(HANDLE);
```

- **Empty diamond:** The empty diamond operator reads from all files specified on the command line (in order).

```
0  while (<>) {  
1    print;  
2  }
```

- **Slurp mode:** We can temporarily make the input separator undefined, then read a file.

```
0  open(my $file, "<", "file.txt")  
1    or die "$!";  
2  
3  my $contents = do {  
4    local $/  
5    <$file>;  
6  }
```

This is called **file slurping**

Subroutines

- Creating and calling subroutines

```
0  sub fn {  
1      ...  
2  }  
3  
4  fn(); # Preferred method of calling  
5  &fn();  
6  &fn;  
7  fn;
```

The older ampersand syntax bypasses any defined prototypes and implicitly passes the caller's current `@_` argument array.

You can pass arguments to a subroutine by including them in the parentheses when you call it. The arguments end up in a special array called `@_` which is only visible inside the subroutine.

- **Shift and unshift:** The Perl shift function removes and returns the first element of an array. This shortens the array by one and shifts all subsequent elements to the left. If the target array is empty, it returns undef

The unshift function adds one or more elements directly to the beginning of an array. It is the exact opposite of the shift function, which removes an item from the front.

```
0  sub print_headers {  
1      my $title = shift "Untitled";  
2      my $author = shift "Anonymous";  
3      print "$title\n"  
4      print "by\n"  
5      print "$author\n";  
6  }
```

- **Passing arrays and hashes:** To pass in a single array or hash to a subroutine, make it the final element in your argument list.

Passing in more than one array or hash causes problems. This is because of list flattening. When Perl sees a number of items in parentheses these are combined into one big list.

```
0  # Flatten two lists into one big list and put that in an  
    ↪ array  
1  my @biglist = (@list1, @list2);  
2  
3  # Flatten (join) two hashes into one big list an put that in  
    ↪ a hash  
4  my %bighash = (%hash1, %hash2);
```

```

0  my @colours = qw/red blue white green pink/;
1  my @chosen = qw/red white green/;
2
3  print_unchosen(@chosen, @colours);
4
5  sub print_unchosen {
6      my (@chosen, @colours) = @_;
7
8      # at this point @chosen contains:
9      # (red white green red blue white green pink)
10     # and @colours contains () - the empty list.
11 }

```

We can avoid this problem by using references:

```

0  my @colours = qw/red blue white green pink/;
1  my @chosen = qw/red white green/;
2  print_unchosen(\@chosen, \@colours);
3  sub print_unchosen {
4      my ($chosen, $colours) = @_;
5      my @chosen = @$chosen;
6      my @colours = @$colours;
7      # at this point @chosen contains:
8      # (red white green)
9      # and @colours contains (red blue white green pink)
10 }

```

Regular expressions

- **Matching operator:** The most basic regular expression operator is the matching operator, `m/PATTERN/`.
 - Works on `$_` by default.
 - In scalar context, returns true (1) if the match succeeds, or false (the empty string) if the match fails.
 - In list context, returns a list of any parts of the pattern which are enclosed in parentheses. If there are no parentheses, the entire pattern is treated as if it were parenthesised.
 - The `m` is optional if you use slashes as the pattern delimiters.
 - If you use the `m` you can use any delimiter you like instead of the slashes. This is very handy for matching on strings which contain slashes, for instance directory names or URLs.
 - Using the `/i` modifier on the end makes it case insensitive.

```
0  while (<>) {  
1      print if m/foo/;  
2  }
```

- **s/pattern/replacement:** This is the substitution operator, and can be used to find text which matches a pattern and replace it with something else.
 - Works on `$_` by default.
 - In scalar context, returns the number of matches found and replaced.
 - In list context, behaves the same as in scalar context and returns the number of matches found and replaced (a cause of more than one mistake...).
 - You can use any delimiter you want, the same as the `m//` operator.
 - Using `/g` on the end of it matches globally, otherwise matches (and replaces) only the first instance of the pattern.
 - Using the `/i` modifier makes it case insensitive.
- **Binding operators:** If we want to use `m//` or `s///` to operate on something other than `$_` we need to use binding operators to bind the match to another string.
 - `=~` True if the pattern matches
 - `!~` True if the pattern doesn't match

The most common use is to test if a string matches a regular expression. It returns a true value if it matches, and false if it doesn't.

```
0  my $text = "The quick brown fox jumps over the lazy dog";  
1  
2  # Evaluates to true because "fox" is in the string  
3  if ($text =~ /fox/) {  
4      print "Found a fox!\n";  
5  }
```

You can use `=~` with the `s///` operator to find a pattern and replace it. This modifies the original variable on the left

```
0 my $greeting = "Hello World";
1
2 # Replaces "World" with "Perl"
3 $greeting =~ s/World/Perl/;
4
5 print $greeting; # Outputs: Hello Perl
```

When you use matching parentheses `()` in your regular expression, `=~` can be used to extract those specific pieces into variables.

```
0 my $date = "2026-08-20";
1
2 # Captures the digits into 1,2, and 3
3 if ($date =~ /(\d{4})-(\d{2})-(\d{2})/) {
4     print "Year: $1, Month: $2, Day: $3\n";
5 }
```

does not match a pattern, use the `!~` operator instead.

```
0 my $email = "user-at-domain.com";
1
2 if ($email !~ /@/) {
3     print "Invalid email: Missing '@' symbol.\n";
4 }
```

In Perl, `$1`, `$2`, `$3` (and so on) are special variables that hold captured text from the most recent successful regular expression match.

- **Modifiers:**

- `/i` Make match/substitute match case insensitive
- `/g` Make substitute global (all occurrences are changed)
- `/m` – Multi-line mode. Causes `^` and `$` to match the start and end of any line within the string rather than just the absolute string boundaries.
- `/s` – Single-line mode. Treats the string as a single line, allowing the dot `(.)` metacharacter to match newline `(\n)` characters.
- `/x` – Extended legibility. Tells Perl to ignore unescaped whitespace and allow comments `(#)` inside the regex pattern for clearer layout formatting
- `/e` – Evaluate replacement. Treats the replacement side as a block of Perl code to execute, utilizing its return value as the final replacement text
- `/ee` – Double evaluate. Evaluates the string on the replacement side as a Perl string, and then runs `eval` a second time on the result.
- `/r` – Non-destructive substitution. Instead of modifying the original variable, it performs the operation and returns the mutated string copy.

- /a – ASCII-only restrictions. Forces character classes like \d, \s, and \w to strictly match ASCII characters only.
- /u – Unicode rules. Forces character classes to abide by full Unicode rules (the default setting for most modern Perl scripts).
- /l – Locale rules. Instructs Perl to use the underlying system’s runtime locale setting.
- /d – Default rules. Uses old, backwards-compatible platform defaults.

You can also activate or deactivate modifiers dynamically inside a regex pattern using the (?imsx-imsx) synt

- (?i)apple matches "Apple" or "APPLE" inline.
- (?-i)apple turns off case-insensitivity from that point forward.

- **Meta characters:**

- ^ Start of string
- \$ End of string
- . Any single character except \n
- \n Newline
- \t Matches a tab
- \s Any whitespace character, such as space, tab, or newline
- \S Any non-whitespace character
- \d Any digit (0 to 9)
- \D Any non-digit
- \w Any "word" character - alphanumeric plus underscore (_)
- \W Any non-word character
- \b A word break - the zero-length point between a word character (as defined above) and a non-word character.
- \B A non-word break - anything other than a word break

- **Quantifiers**

- ? 0 or 1
- * 0 or more
- + 1 or more
- {n} match exactly n times
- {n,} match n or more times
- {n,m} match between n and m times

- **Character classes:** Created the usual way with brackets

- ^ negates the class
- - specifies a range of characters. To mach a literal -, it must be either the first or the last character in the class.
- \$, ., (), {}, *, + and other meta characters taken literally

- **Match variable (\$&):** This special variable holds whatever the last regex matched.

- **Alternation**

Idioms

- **Representation of true and false:** Perl does not have a native true or false keyword. Instead, it determines a value's truth by evaluating what it is.

Only a few specific values are considered false in Perl:

- The number 0 (or 0.0)
- The string "0"
- The empty string ""
- undef (undefined)
- An empty list ()

Everything else is true. When you use a single logical NOT (!), Perl evaluates the operand's truth and returns an absolute boolean result.

- If you pass it something false, it returns 1 (Perl's standard "true" value).
 - If you pass it something true, it must return a "false" value. By language design, Perl's logical operators return the empty string ("") to represent false.
- **Goatse operator =()**: Used to force an expression on its right-hand side into a list context while evaluating the entire assignment in a scalar context. It returns the number of items generated by the right-hand side expression

It is most frequently used to count regex matches in global matching mode (/g) without storing all the matched text in memory:

```
0 my $text = "cat dog cat bird cat";
1 my $count = () = $text =~ /cat/g;
2 print $count; # Outputs 3
```

- **Bang bang:** The bang bang operator forces any scalar value into a strict boolean 1 or empty string "" (which Perl treats as false). It is simply two consecutive logical NOT (!) operators.

The first ! evaluates the value and flips its truthiness to a strict boolean (making a true value "" and a false value 1). The second ! flips it back.

Perl doesn't have a dedicated boolean type. If a function returns an object, a string, or a number on success, using !! cleans up the data so it only returns a clean 1 (true) or "" (false).

```
0 my $user_input = "Hello";
1
2 my $pure_boolean = !!$user_input;
3 print $pure_boolean; # Outputs 1
```

Perl